

Trabajo Practico Integrador N. 3

JavaScript, consumo de datos.

Fecha de entrega: 30 de junio

1. Fundamentacion

Este trabajo practico integrador da continuidad al TP2. A partir de la estructura HTML y los estilos ya desarrollados, cada estudiante debera incorporar JavaScript para transformar el sitio en una interfaz dinamica capaz de consumir datos desde una API fake. El objetivo no es desarrollar un backend real, sino comprender el flujo basico de una aplicacion web moderna: obtener datos externos, procesarlos y mostrarlos en la interfaz.

El modelo de referencia provisto utiliza un archivo JSON local como API fake y un script JavaScript que realiza la consulta mediante fetch. El archivo api-fake.json define productos con campos como title, description, price, category, pictures y tags. El ejemplo productos.js muestra una implementacion asincronica con fetch, headers, try/catch y renderizado dinamico en el contenedor de productos.

2. Objetivos de aprendizaje

- Incorporar JavaScript al sitio entregado en el TP2.
- Consumir datos desde una API fake utilizando fetch.
- Procesar datos en formato JSON y renderizarlos dinamicamente en el DOM.
- Reemplazar contenido estatico por contenido generado desde JavaScript.
- Implementar controles basicos de carga, error y estado vacio.
- Agregar interacciones simples, como filtros, busqueda, ordenamiento y newsletter simulado.
- Diferenciar entre una interfaz funcional con datos simulados y una aplicacion conectada a servicios reales.

3. Consigna general

Tomar como punto de partida la entrega realizada en el TP2 y adaptarla para que la información principal del sitio sea cargada de manera dinámica desde JavaScript, consumiendo el archivo api-fake.json provisto por la cátedra.

La información de productos, tarjetas, listados, destacados, categorías u otros elementos repetitivos no debe quedar hardcodeada en el HTML. Debe generarse desde JavaScript a partir de los datos disponibles en la API fake.

4. Recursos provistos por la cátedra

- Archivo api-fake.json: fuente de datos simulada.
- Archivo productos.js: ejemplo base de consumo de datos con fetch.
- Archivo listado.html: ejemplo mínimo de página con contenedor preparado para recibir datos dinámicos.
- Archivo styles.css: estilos de referencia para listado de productos.
- Entrega del TP2: estructura visual y navegación base que cada estudiante deberá evolucionar.

5. Modelo de API fake

La API fake será un archivo JSON local. Puede ubicarse en una carpeta api, data o assets, según la organización del proyecto. Ejemplo sugerido:

/api-fake.json

Cada elemento del JSON representa un producto o recurso comercial. El modelo mínimo esperado es:

```
{
  "id": 623,
  "title": "Jabon liquido de lavanda",
  "description": "Descripcion del producto",
  "price": 1.2,
  "category_id": 195,
  "pictures": ["https://url-de-imagen.jpg"],
  "category": {
    "id": 195,
    "title": "Verduras",
    "description": "Descripcion de categoria"
  },
  "tags": [
    { "id": 101, "title": "Promocion" }
  ]
}
```

6. Requisitos funcionales obligatorios

Requisito	Descripcion
RF1. Consumo de datos	El sitio debe consultar api-fake.json mediante fetch. No se aceptara copiar manualmente los productos en el HTML.
RF2. Renderizado dinamico	Los productos o elementos principales del sitio deben generarse con JavaScript usando createElement, template strings o una estrategia equivalente.
RF3. Estado de carga	Mientras se cargan los datos, debe mostrarse un mensaje visible, por ejemplo: Cargando productos...
RF4. Control de errores	Debe implementarse try/catch o catch para mostrar un mensaje si los datos no pueden cargarse.
RF5. Uso de async/await	La solucion principal debe usar una funcion asincronica para obtener la informacion.
RF6. Vinculacion con todas las paginas relevantes	Las paginas generadas en el TP2 que muestren informacion repetitiva deberan poblarse desde JavaScript o reutilizar funciones comunes.
RF7. Newsletter simulado	El formulario de newsletter debe ser funcional en la interfaz: validar datos, mostrar mensaje de exito o error y guardar temporalmente la suscripcion en localStorage. No debe enviar informacion a un servicio real.
RF8.Codigo organizado	El codigo JavaScript debe estar separado en archivos .js y correctamente vinculado desde HTML.
RF9. Compatibilidad visual	La incorporacion de JavaScript no debe romper el diseno construido en el TP2.

7. Funcionalidades sugeridas

Ademas de los requisitos obligatorios, se recomienda incorporar al menos dos de las siguientes funcionalidades:

- Buscador por nombre o descripcion.
- Filtro por categoria.
- Filtro por tag, por ejemplo Promocion u Organico.
- Ordenamiento por precio ascendente o descendente.
- Contador de productos encontrados.
- Vista de productos destacados.
- Boton para limpiar filtros.
- Detalle simple de producto usando modal, seccion expandida o pagina de detalle simulada.
- Carrito simulado con localStorage, sin proceso de pago real.

8. Newsletter simulado

El newsletter debe comportarse como una funcionalidad real desde el punto de vista del usuario, aunque no exista un servicio detras. Se espera que el formulario:

- Solicite al menos un correo electronico.
- Valide que el campo no este vacio y que tenga formato basico de email.
- Muestre un mensaje de confirmacion cuando la suscripcion sea correcta.
- Guarde el correo en localStorage para simular persistencia.
- Evite registrar dos veces el mismo correo o avise si ya estaba suscripto.

```
const formNewsletter = document.querySelector("#newsletter-form");
const inputEmail = document.querySelector("#newsletter-email");
const mensajeNewsletter = document.querySelector("#newsletter-mensaje");

formNewsletter.addEventListener("submit", function(event) {
  event.preventDefault();

  const email = inputEmail.value.trim();
  if (!email.includes("@")) {
    mensajeNewsletter.textContent = "Ingresá un correo válido.";
    return;
  }

  const suscriptores = JSON.parse(localStorage.getItem("newsletter")) || [];
  if (suscriptores.includes(email)) {
    mensajeNewsletter.textContent = "Este correo ya está suscripto.";
    return;
  }

  suscriptores.push(email);
  localStorage.setItem("newsletter", JSON.stringify(suscriptores));
  mensajeNewsletter.textContent = "Suscripción registrada correctamente.";
  formNewsletter.reset();
});
```

9. Estructura minima sugerida del proyecto

```
/tp3
index.html
productos.html
contacto.html
/css
  styles.css
/js
  main.js
  productos.js
  newsletter.js
/data
  api-fake.json
/img
  imagenes-del-sitio
```

10. Ejemplo base de consumo con fetch

El siguiente ejemplo puede ser usado como referencia. Cada estudiante deberá adaptarlo a su propio TP2.

```
const endpoint = "../data/api-fake.json";
const contenedor = document.querySelector("#productos");
const mensaje = document.querySelector("#mensaje");

function crearCardProducto(producto) {
  const imagen = producto.pictures && producto.pictures.length > 0
    ? producto.pictures[0]
    : "../img/placeholder.jpg";

  return `
    <article class="producto">
      
      <h2>${producto.title}</h2>
      <p>${producto.description}</p>
      <p>Categoría: ${producto.category?.title || "Sin categoría"}</p>
      <strong>${producto.price}</strong>
    </article>
  `;
}

function renderizarProductos(productos) {
  contenedor.innerHTML = "";
  mensaje.textContent = "";

  if (productos.length === 0) {
    mensaje.textContent = "No se encontraron productos.";
    return;
  }

  productos.forEach(producto => {
    contenedor.innerHTML += crearCardProducto(producto);
  });
}

async function obtenerProductos() {
  try {
    mensaje.textContent = "Cargando productos...";

    const response = await fetch(endpoint, {
      headers: { "accept": "application/json" }
    });

    if (!response.ok) {
      throw new Error("No se pudo obtener la información.");
    }

    const data = await response.json();
    renderizarProductos(data);
  } catch (error) {
    mensaje.textContent = "No se pudieron cargar los productos.";
    console.error(error);
  }
}

obtenerProductos();
```

11. Condiciones de entrega

- Fecha limite de entrega: 30 de junio.
- Entregar el proyecto completo comprimido o mediante repositorio, segun indique la catedra.
- El sitio debe poder ejecutarse localmente.
- Se recomienda ejecutar el proyecto con Live Server o un servidor local equivalente, ya que fetch sobre archivos locales puede fallar si se abre el HTML directamente desde file://.
- No se deben utilizar servicios reales para newsletter, pagos, login o envio de formularios.
- El codigo debe estar ordenado y comentado en las partes relevantes.

12. Criterios de evaluacion

Criterio	Peso	Descripcion
Consumo de API fake con fetch	20%	Uso correcto de fetch, async/await, lectura de JSON y control basico de errores.
Renderizado dinamico del contenido	20%	El contenido principal se genera desde JavaScript y no queda hardcodeado en HTML.
Integracion con el TP2	15%	La solucion respeta y mejora la estructura visual y navegacion ya desarrollada.
Interactividad adicional	15%	Incluye busqueda, filtros, ordenamiento, contador, detalle, carrito simulado u otra funcionalidad equivalente.
Newsletter simulado	10%	Formulario funcional con validacion, mensajes y almacenamiento local.
Organizacion del codigo	10%	Archivos separados, nombres claros, funciones reutilizables y ausencia de codigo innecesariamente duplicado.
Presentacion y funcionamiento general	10%	El sitio funciona localmente, no presenta errores visibles y mantiene coherencia visual.

13. Rubrica sintetica

Nivel	Descripcion
Excelente	Cumple todos los requisitos, incorpora interacciones adicionales, maneja errores y mantiene codigo claro y reutilizable.
Bueno	Cumple los requisitos principales, consume la API fake y renderiza correctamente, con pequenas oportunidades de mejora.

Basico	Consume datos y muestra contenido, pero con baja organizacion, poca reutilizacion o controles incompletos.
Insuficiente	No consume la API fake, mantiene contenido hardcodeado o el sitio no funciona correctamente.

14. Observaciones para el estudiante

- El objetivo del TP no es que el sitio tenga backend real, sino que simule el comportamiento de una aplicacion dinamica.
- No se evaluara la cantidad de productos cargados, sino la correcta integracion entre JavaScript, JSON y DOM.
- El archivo api-fake.json puede ser ampliado por el estudiante siempre que respete una estructura coherente.
- Si una pagina del TP2 tenia contenido repetitivo, se espera que en el TP3 ese contenido sea cargado dinamicamente.
- El newsletter debe mostrar comportamiento funcional, aunque sus datos se almacenen solo en localStorage.

15. Entrega esperada

La entrega final debe incluir un sitio funcional basado en el TP2, con JavaScript incorporado, consumo de api-fake.json y al menos una funcionalidad de interaccion simulada. El proyecto debe poder revisarse localmente sin depender de servicios externos reales.